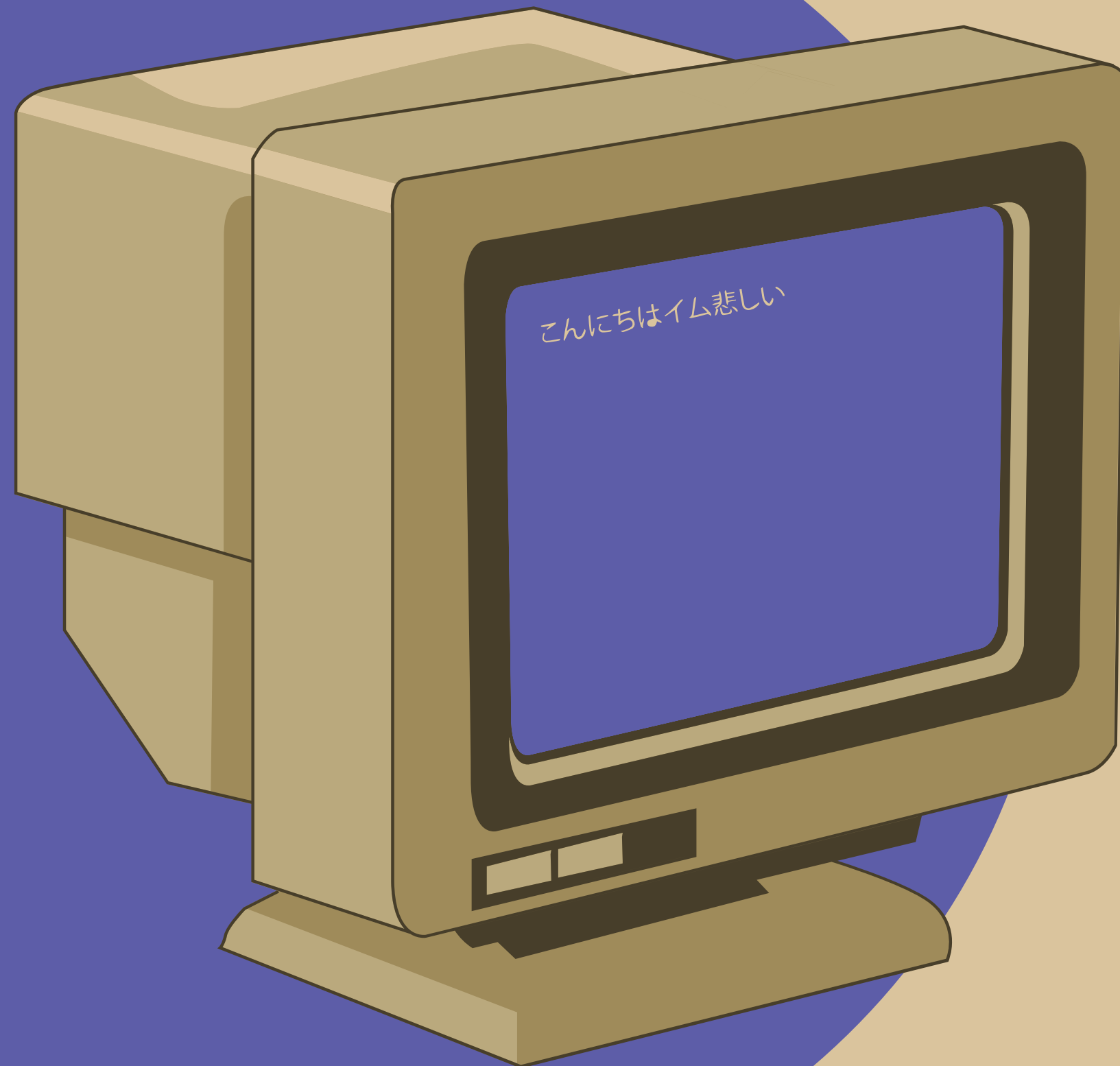
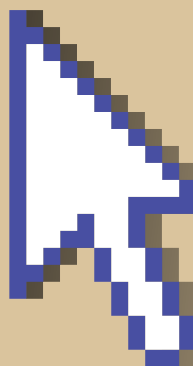




IMPLEMENTAÇÃO EM HARDWARE DE UM ALGORITMO DE SUBSTITUIÇÃO DE CACHE BASEADO EM PERCEPTRON

FERNANDO INFANTINI
RONALDY GELOS

PROJETO INTEGRADOR 4
PROFESSOR: BRUNO NEVES



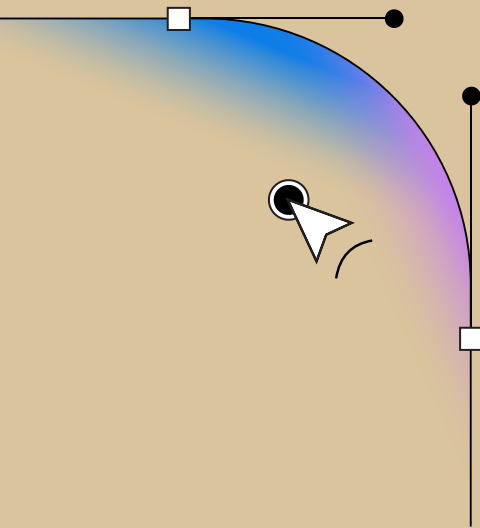
PROBLEMA E MOTIVAÇÃO

- O acesso à memória principal é lento, por isso caches eficientes são essenciais para o desempenho do processador;
- Limitações do LRU tradicional:
 - Bom para padrões de localidade temporal simples.
 - Falha em padrões complexos (ex: varreduras longas, matrizes).
- Hipótese: Redes neurais simples podem aprender padrões de acesso em tempo de execução para prever com mais precisão qual bloco descartar?

PERCEPTRON

- O que é o perceptron: Um classificador linear que atribui notas para as vias da cache;
- Como essa substituição funciona:
 - Cada índice possui pesos globais atualizados a cada acerto/erro.
 - As características do bloco (Tag, Reuso) são multiplicadas pelos pesos.
 - Campeonato de penalidades: a via com a menor nota é a vítima escolhida para o descarte.

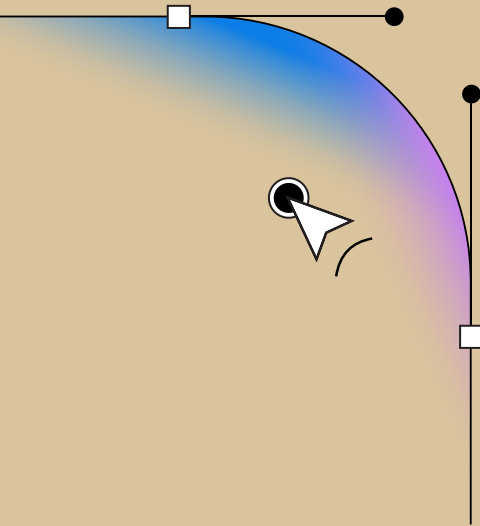
$$y = w_0 + \sum_{i=1}^n x_i w_i$$



TRABALHOS RELACIONADOS

Dynamic Branch Prediction with Perceptrons (Jiménez & Lin, 2001)

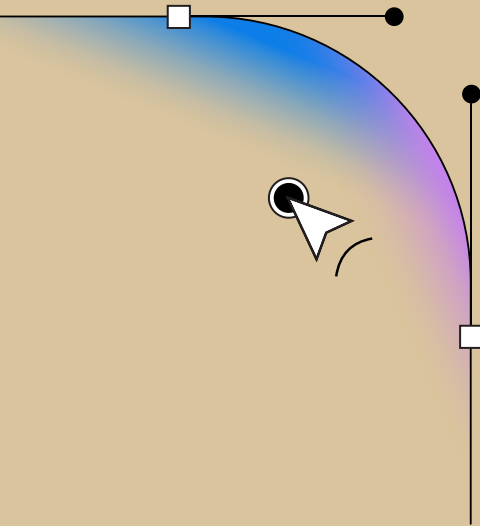
- Contribuição: Substituição dos tradicionais contadores de 2 bits por redes neurais simples (perceptrons) para a previsão de desvios no processador.
- O que extraímos de interessante:
 - Escalabilidade: O custo de hardware cresce de forma linear com o tamanho do histórico, superando o crescimento exponencial dos modelos antigos.
 - Quebra de Paradigma: Provou que redes neurais, quando simplificadas, são viáveis e rápidas o suficiente para rodar no nível de hardware (pipeline), entregando 10% a mais de precisão.



TRABALHOS RELACIONADOS

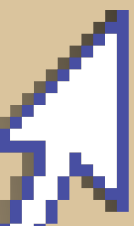
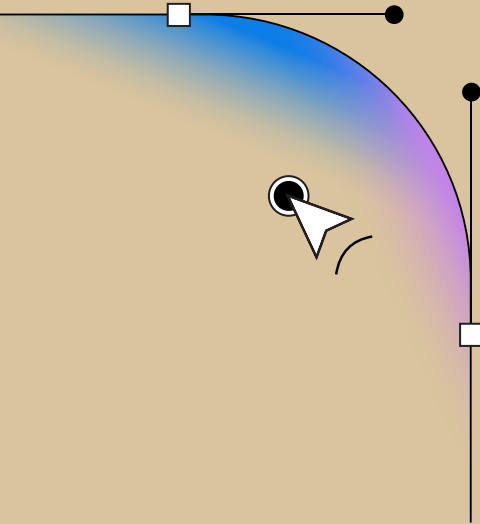
Perceptron Learning for Reuse Prediction (Jiménez et al., 2016)

- Contribuição: Adaptação da lógica do perceptron para gerenciar políticas de substituição e bypass em Caches de Último Nível (LLC).
- O que extraímos de interessante:
 - Mitigação de Interferência: O uso de tabelas independentes para múltiplas características (como o endereço do bloco) elimina a interferência destrutiva de métodos anteriores.
 - Evolução da Técnica: Mostra o amadurecimento da ideia de 2001. A mesma base matemática foi expandida para reduzir as taxas de falsos positivos no cache em 50%.



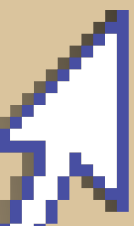
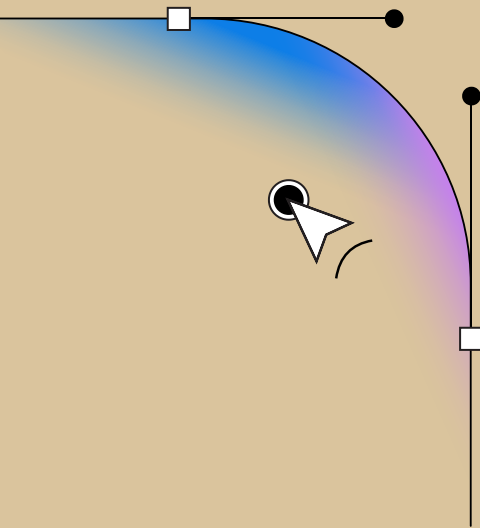
AMBIENTE DE SIMULAÇÃO E CARGAS DE TRABALHO

- Desenvolvimento de um simulador de hierarquia de cache em linguagem C.
- Baseline (Linha de Base): Implementação da política LRU baseada no algoritmo fornecido pelo professor.
- Bateria de Testes: Utilização dos benchmarks padrão da disciplina + Inclusão de um novo benchmark de Convolução de Matrizes.



O MODELO PERCEPTRON

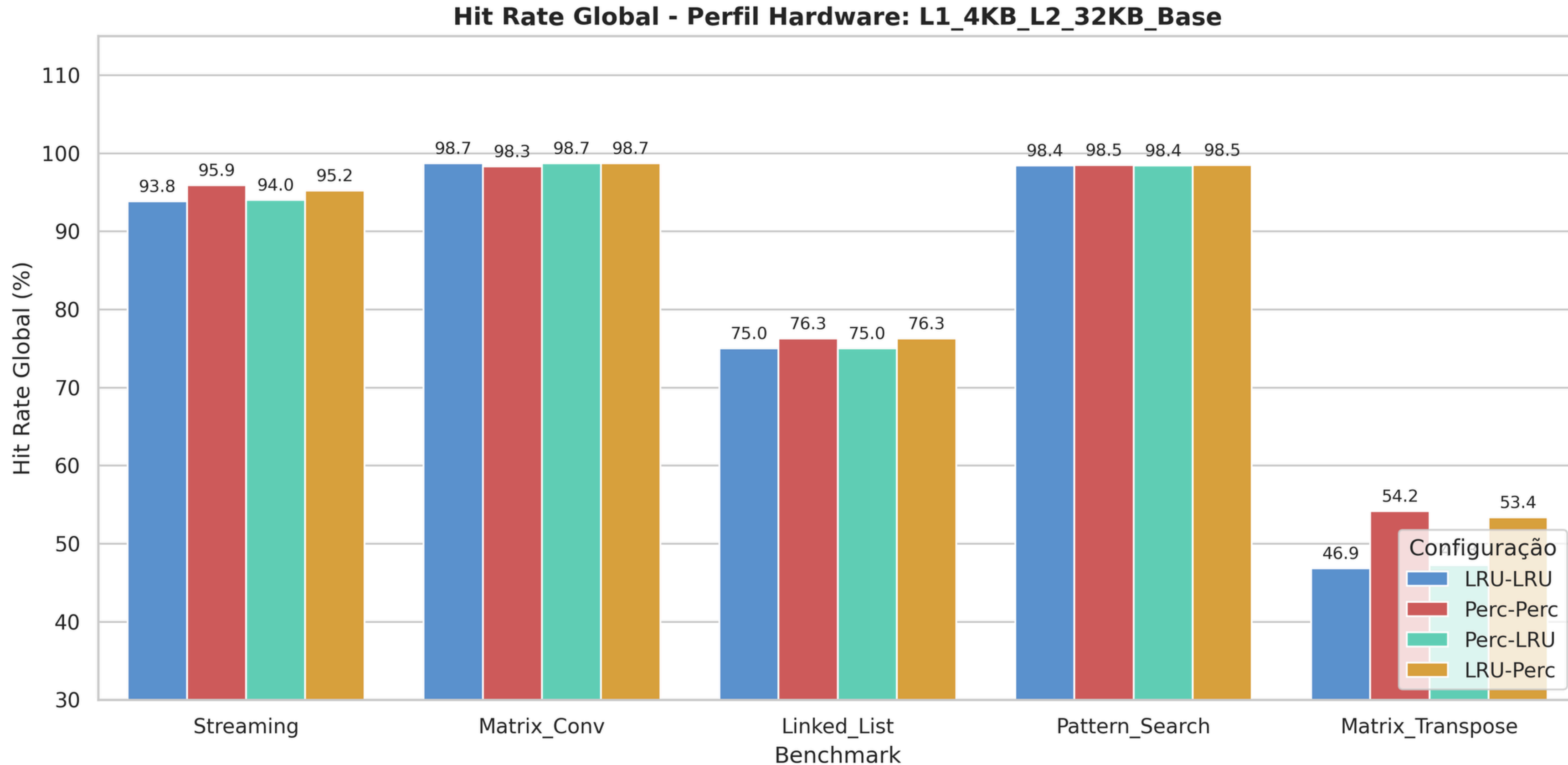
- Transição de olhar apenas para o passado (LRU) para prever o futuro (Perceptron).
- Extração de Features: Conversão de metadados da linha de cache (Tags e Histórico de reuso).
- Vetorização Bipolar: Transformação das entradas estritamente em +1 (True) ou -1 (False).



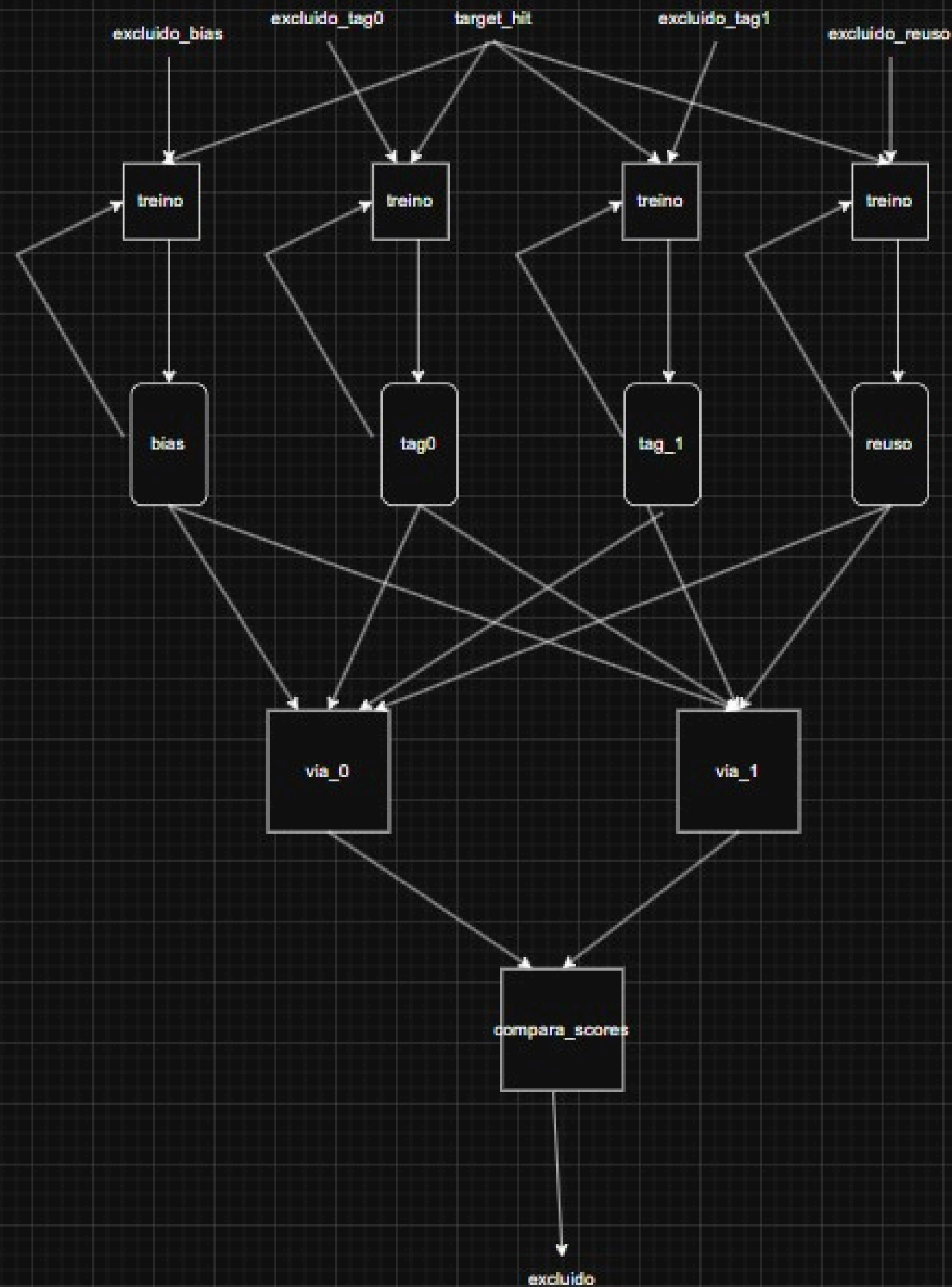
RESULTADOS DA ETAPA DE SOFTWARE

Benchmark	Configuração (L1 - L2)	Hit Rate L1 (%)	Hit Rate L2 (%)	Hit Rate Global (%)
<i>Linked List</i>	LRU - LRU	49,99	49,99	74,99
	LRU - Perceptron	49,99	52,53	76,26
	Perceptron - LRU	49,99	49,99	74,99
	Perceptron - Perceptron	49,99	52,53	76,26
<i>Matrix Convolution</i>	LRU - LRU	97,44	49,98	98,72
	LRU - Perceptron	97,44	49,98	98,72
	Perceptron - LRU	88,66	88,70	98,72
	Perceptron - Perceptron	88,66	84,90	98,29
<i>Matrix Transpose</i>	LRU - LRU	43,75	5,56	46,87
	LRU - Perceptron	43,75	17,10	53,37
	Perceptron - LRU	43,81	6,16	47,27
	Perceptron - Perceptron	43,81	18,47	54,19
<i>Pattern Search</i>	LRU - LRU	96,87	49,99	98,44
	LRU - Perceptron	96,87	50,92	98,47
	Perceptron - LRU	96,87	49,99	98,44
	Perceptron - Perceptron	96,87	50,92	98,47
<i>Streaming</i>	LRU - LRU	87,69	49,97	93,84
	LRU - Perceptron	87,69	61,18	95,22
	Perceptron - LRU	88,02	49,89	94,00
	Perceptron - Perceptron	88,02	65,96	95,92

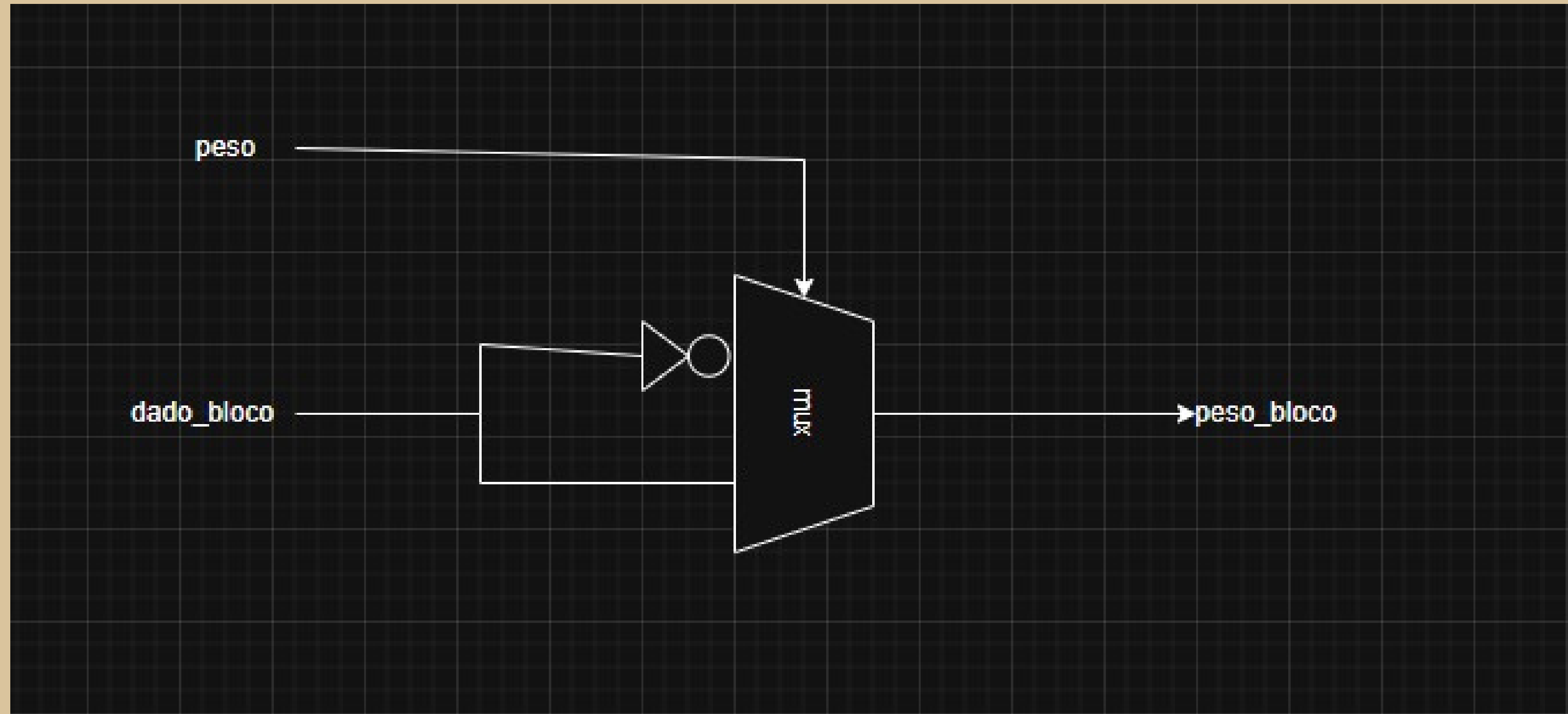
RESULTADOS DA ETAPA DE SOFTWARE



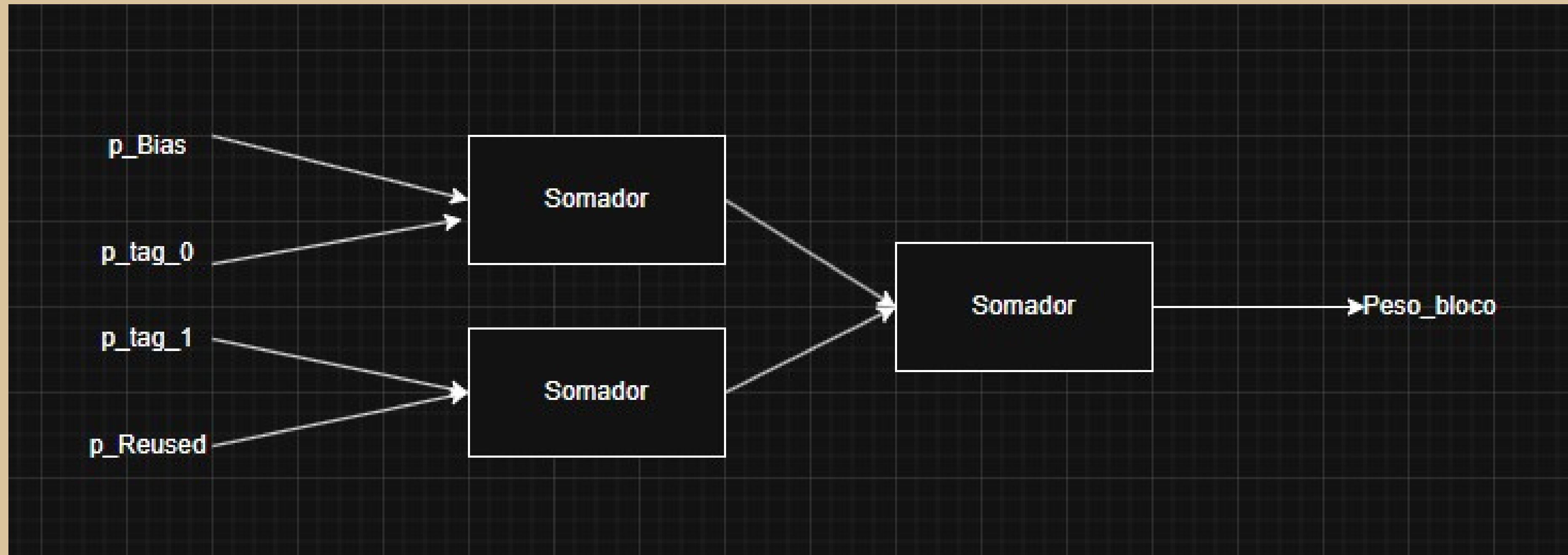
RTL PERCEPTRON



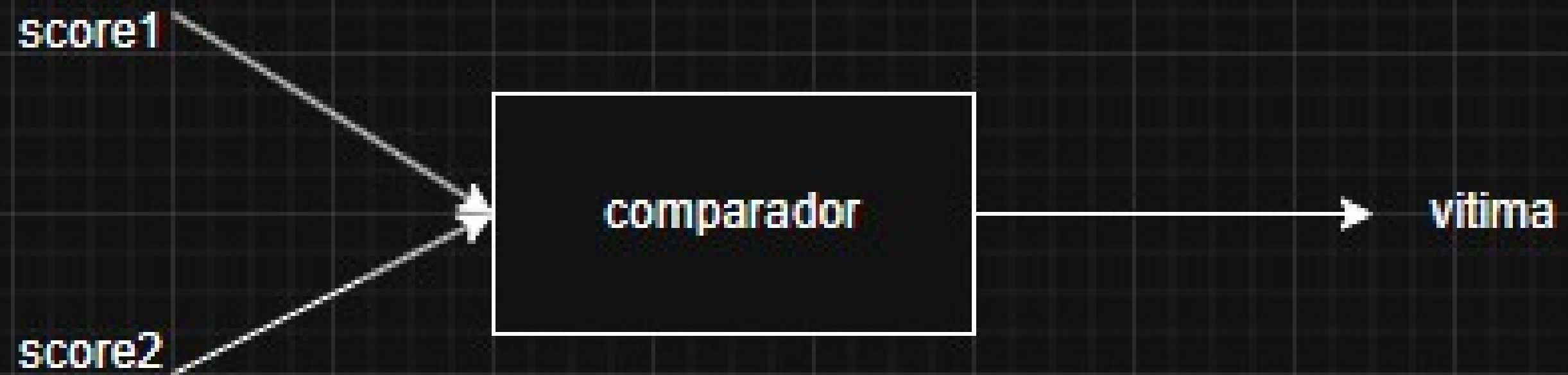
RTL PERCEPTRON



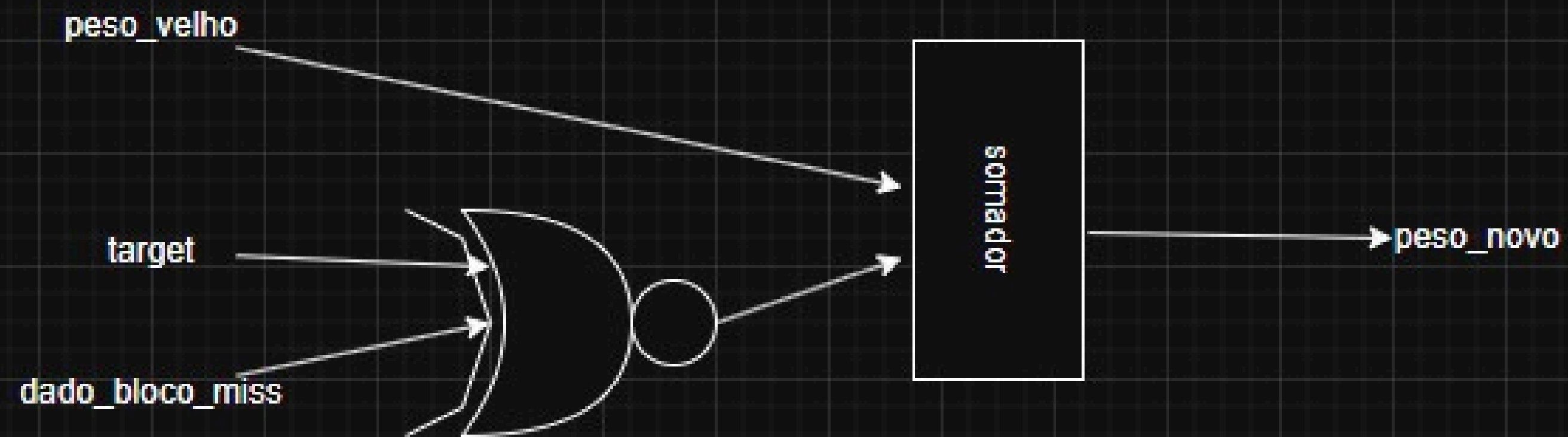
RTL PERCEPTRON



RTL PERCEPTRON

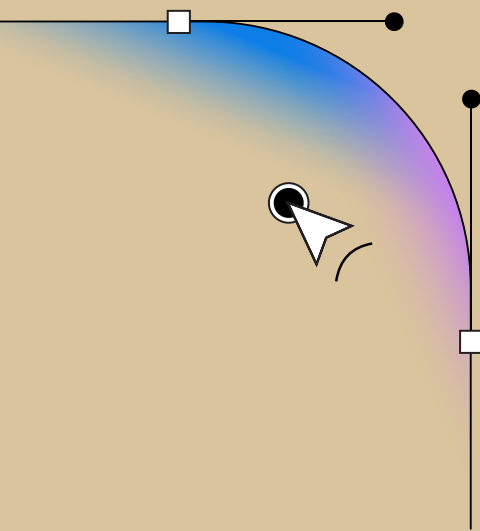


RTL PERCEPTRON



IMPLEMENTAÇÃO EM VELIROG

- Parametrização da cache para permitir criação de diferentes níveis de cache a partir dos mesmos arquivos



```
module cache #(
    parameter ADDRESS_SIZE = 32,
    parameter DATA_SIZE = 32,    // Tamanho da palavra em bits
    parameter BLOCK_SIZE = 4,    // Palavras por bloco
    parameter ASSOCIATIVITY = 4, // Numero de vias
    parameter N_BLOCKS = 256,    // Blocos por via
    parameter ALGORITHM = 1      // 0 para LRU, 1 para Perceptron
);
```


IMPLEMENTAÇÃO EM VELIROG

- Arvore de somadores para realizar calculos de score em paralelo
- O laço for foi utilizado pois ma sintese realiza o seu conteúdo em paralelo

```
for (k = 0; k < ASSOCIATIVITY; k = k + 1) begin
    // Produto Escalar adaptado para features binárias
    score[k] = w_bias[index] +
        (way_tag0[index][k] ? w_tag0[index] : -w_tag0[index]) +
        (way_tag1[index][k] ? w_tag1[index] : -w_tag1[index]) +
        (way_reused[index][k] ? w_reused[index] : -w_reused[index]);

    // A via com menor pontuação (menor chance de reuso) vira a vítima
    if (score[k] < min_score) begin
        min_score = score[k];
        best_way = k;
    end
end
```

IMPLEMENTAÇÃO EM VELIROG

- Treinamento dos pesos ocorrendo de forma otimizada utilizando somas ou subtrações de 1 em 1

```
always @(posedge clock) begin
  if (enable) begin
    begin: training_block
      integer j;
      if (!hits) begin
        // CENÁRIO A: HIT
        for (j = 0; j < ASSOCIATIVITY; j = j + 1) begin
          if (hits[j]) begin
            way_reused[index][j] <= 1'b1;
            way_tag0[index][j] <= curr_tag0;
            way_tag1[index][j] <= curr_tag1;

            if (w_bias[index] < 31) w_bias[index] <= w_bias[index] + 1;
            if (way_tag0[index][j] < 31) w_tag0[index] <= w_tag0[index] + 1; end else begin if (w_tag0[index] > -32) w_tag0[index] <= w_tag0[index] - 1; end
            if (way_tag1[index][j] < 31) w_tag1[index] <= w_tag1[index] + 1; end else begin if (w_tag1[index] > -32) w_tag1[index] <= w_tag1[index] - 1; end
            if (way_reused[index][j]) begin if (w_reused[index] < 31) w_reused[index] <= w_reused[index] + 1; end else begin if (w_reused[index] > -32) w_reused[index] <= w_reused[index] - 1; end
          end
        end
      end else begin
        // CENÁRIO B: MISS
        for (j = 0; j < ASSOCIATIVITY; j = j + 1) begin
          if (victims[j]) begin
            valid[index][j] <= 1'b1;
            way_reused[index][j] <= 1'b0;
            way_tag0[index][j] <= curr_tag0;
            way_tag1[index][j] <= curr_tag1;

            if (w_bias[index] > -32) w_bias[index] <= w_bias[index] - 1;
            if (way_tag0[index][j] > -32) w_tag0[index] <= w_tag0[index] - 1; end else begin if (w_tag0[index] < 31) w_tag0[index] <= w_tag0[index] + 1; end
            if (way_tag1[index][j] > -32) w_tag1[index] <= w_tag1[index] - 1; end else begin if (w_tag1[index] < 31) w_tag1[index] <= w_tag1[index] + 1; end
            if (way_reused[index][j]) begin if (w_reused[index] > -32) w_reused[index] <= w_reused[index] - 1; end else begin if (w_reused[index] < 31) w_reused[index] <= w_reused[index] + 1; end
          end
        end
      end
    end
  end
end
```

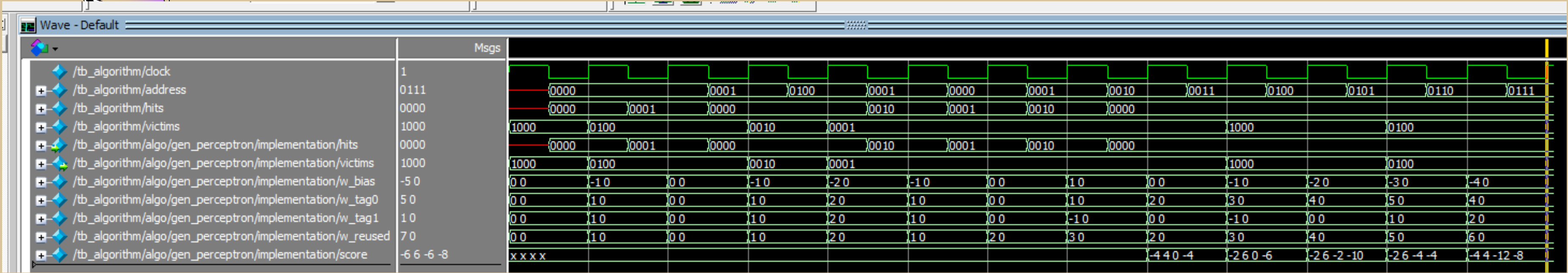
IMPLEMENTAÇÃO EM VELIROG

- Para podermos selecionar de forma mais simples o algoritmo de substituição foi criado um arquivo que realiza a instanciação do algoritmo desejado por meio de um parametro .

```
generate
  if (ALGORITHM == 1) begin : gen_perceptron
    perceptron #(
      .ADDRESS_SIZE(ADDRESS_SIZE),
      .ASSOCIATIVITY(ASSOCIATIVITY),
      .N_BLOCKS(N_BLOCKS)
    ) implementation (
      .address(address),
      .index(index),
      .hits(hits),
      .enable(enable),
      .wr(wr),
      .clock(clock),
      .victims(victims)
    );
  end else begin : gen_lru
    wire [ASSOCIATIVITY-1:0] lru_victims [N_BLOCKS-1:0];
    genvar i;
    for(i=0; i<N_BLOCKS; i=i+1) begin : lru_loop
      wire algo_req = (i == index) ? enable : 1'b0;
      lru #(
        .ADDRESS_SIZE(ADDRESS_SIZE),
        .ASSOCIATIVITY(ASSOCIATIVITY)
      ) lru_impl (
        .address(address),
        .hits(hits),
        .enable(algo_req),
        .wr(wr),
        .clock(clock),
        .victims(lru_victims[i])
      );
    end
    assign victims = lru_victims[index];
  end
endgenerate
```

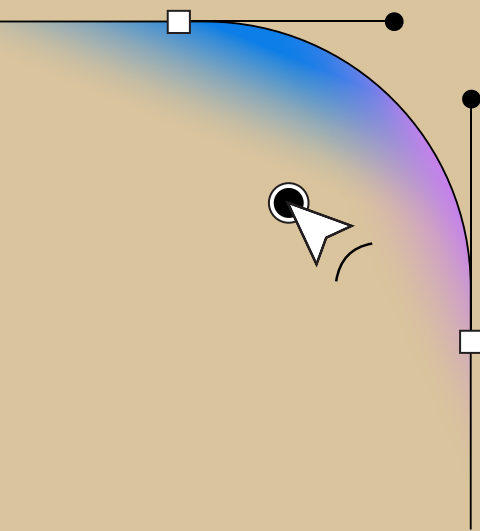
VALIDAÇÃO POR TESTBENCHS

- Para verificar o comportamento de diferentes componentes da cache foram criados testbenches para analisar o comportamento dos sinais ao longo da execução.



CRIAÇÃO DE CACHE DE 2 NÍVEIS

- Devido a criação de arquivos parametrizados isso simplificou a tarefa de criar diferentes topos instanciando o mesmo arquivo com diferentes parametros



```
cache #(
    .ADDRESS_SIZE(32),
    .DATA_SIZE(32),
    .BLOCK_SIZE(4),
    .ASSOCIATIVITY(2),
    .N_BLOCKS(32),
    .ALGORITHM(1)
) cache_l1 (
    .address_in(l1_addr_in),
    .data_in(l1_data_in),
    .req(l1_req),
    .wr(l1_wr),
    .ready(l1_ready),
    .clk(clock),
    .address_out(),
    .data_out(l1_data_out_dummy),
    .ce(),
    .we(),
    .ack(1'b0),
    .clk_out()
);
```

```
cache #(
    .ADDRESS_SIZE(32),
    .DATA_SIZE(32),
    .BLOCK_SIZE(4),
    .ASSOCIATIVITY(8),
    .N_BLOCKS(64),
    .ALGORITHM(1)
) cache_l2 (
    .address_in(l2_addr_in),
    .data_in(l2_data_in),
    .req(l2_req),
    .wr(l2_wr),
    .ready(l2_ready),
    .clk(clock),
    .address_out(),
    .data_out(l2_data_out_dummy),
    .ce(),
    .we(),
    .ack(1'b0),
    .clk_out()
);
```


SÍNTESE VIA QUARTUS II

- Para realizar a síntese do nosso código em verilog utilizamos a ferramenta Quartus II com o FPGA Cyclone III.
- O arquivo utilizado foi um top_cache onde setamos os dados da seguinte forma:

Tabela 5: Parâmetros Estruturais e Configuração das Caches L1 e L2

Parâmetro Verilog	Cache L1	Cache L2
ADDRESS_SIZE	32 bits	32 bits
DATA_SIZE	32 bits	32 bits
BLOCK_SIZE	4 palavras	4 palavras
ASSOCIATIVITY	2 vias	8 vias
N_BLOCKS	32 blocos	64 blocos
Conjuntos (<i>Sets</i>)	16 conjuntos	8 conjuntos
Capacidade Total	512 Bytes	1024 Bytes

SÍNTESE VIA QUARTUS II

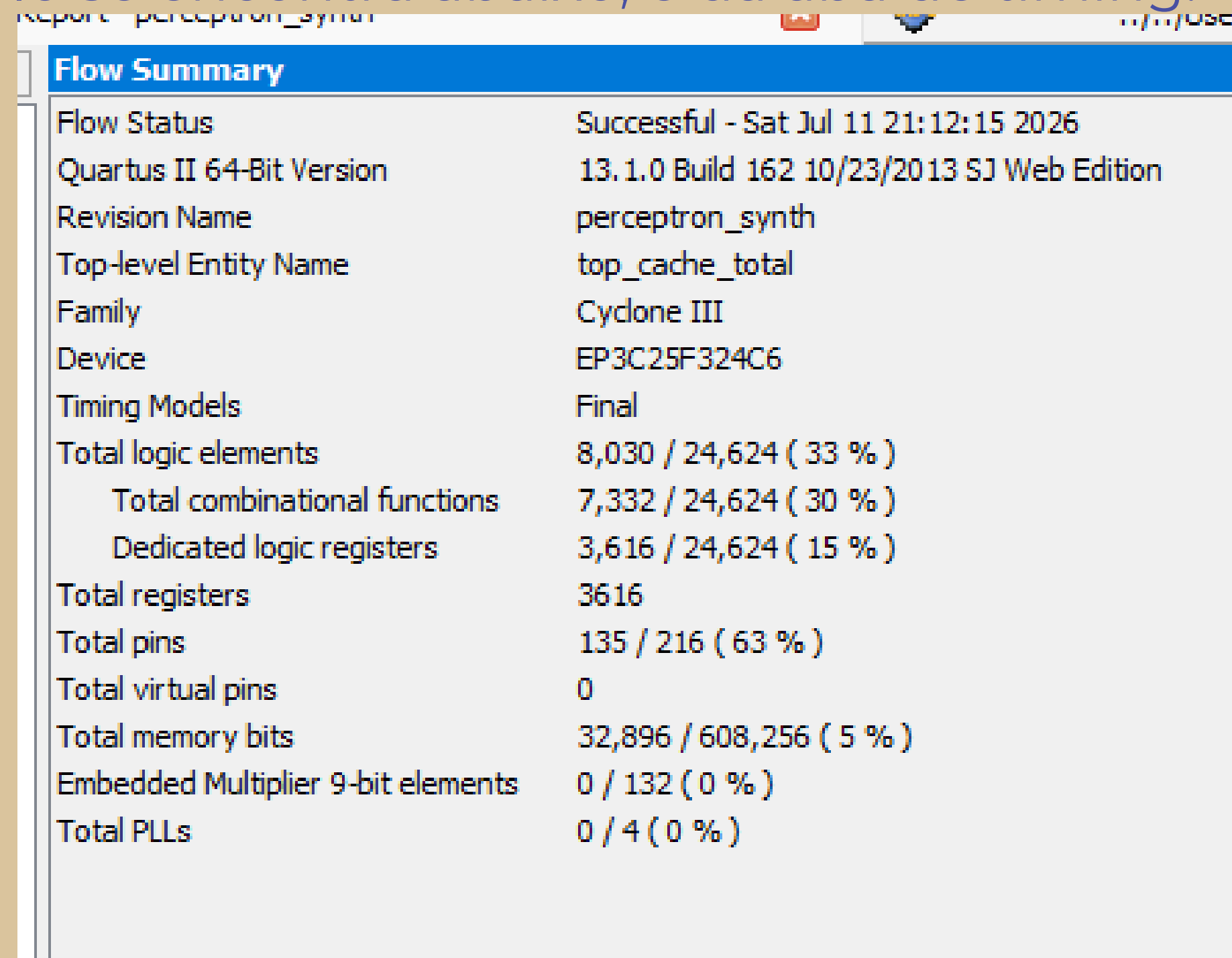
- Ao realizar a síntese com as diferentes configurações de algoritmos de cache obtvemos os seguintes resultados:

Tabela 6: Comparação de Custo Físico no FPGA e Impacto no Timing

Configuração (L1 - L2)	Total de LEs	Registradores	F_{max} (MHz)	BRAMs (bits)
LRU - LRU	4.385	1.824	90,92	32.896
Perceptron - LRU	6.684	2.816	60,02	32.896
LRU - Perceptron	8.030	3.616	30,64	32.896
Perceptron - Perceptron	9.990	4.608	28,93	32.896

SÍNTESE VIA QUARTUS II

- Os dados foram retirados da aba “flow summary”, onde um exemplo se encontra abaixo, e da aba de timing.



Flow Summary	
Flow Status	Successful - Sat Jul 11 21:12:15 2026
Quartus II 64-Bit Version	13.1.0 Build 162 10/23/2013 SJ Web Edition
Revision Name	perceptron_synth
Top-level Entity Name	top_cache_total
Family	Cyclone III
Device	EP3C25F324C6
Timing Models	Final
Total logic elements	8,030 / 24,624 (33 %)
Total combinational functions	7,332 / 24,624 (30 %)
Dedicated logic registers	3,616 / 24,624 (15 %)
Total registers	3616
Total pins	135 / 216 (63 %)
Total virtual pins	0
Total memory bits	32,896 / 608,256 (5 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	0 / 4 (0 %)

INDICE DE EFICIÊNCIA

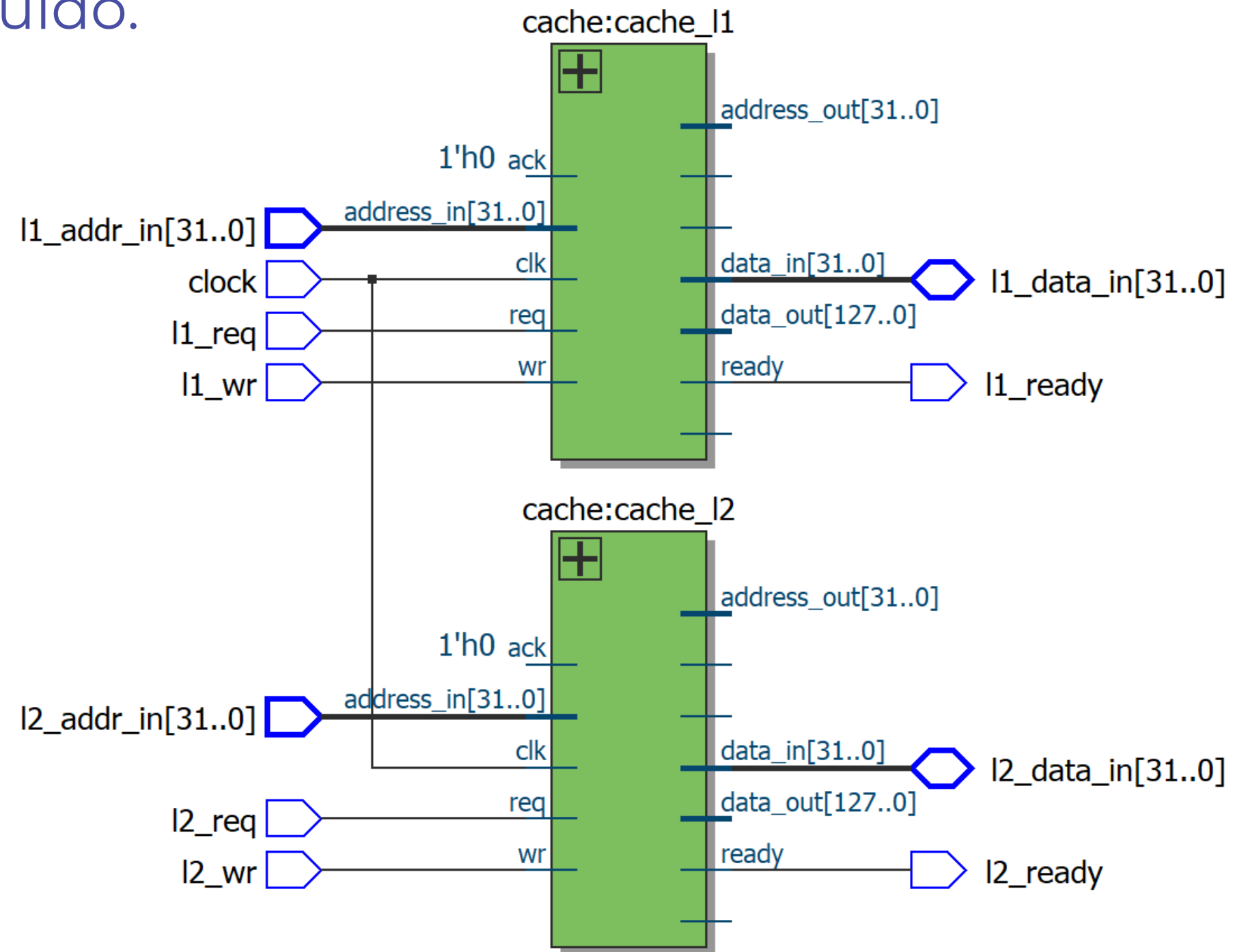
- Também foi realizada um comparação entre a taxa de hits com o ganho de área no FPGA para obter um calculo de custo-beneficio das diferentes configurações da solução.

Tabela 7: Análise do Índice de Eficiência de Hardware (IEC)

Configuração (L1 - L2)	Hit Rate Global	Ganho Hit Rate	LEs Extras	IEC (%/LE)
Perceptron - LRU	82,68%	+0,11%	2.299	0,000048
LRU - Perceptron	84,41%	+1,84%	3.645	0,000504
Perceptron - Perceptron	84,63%	+2,05%	6.463	0,000318

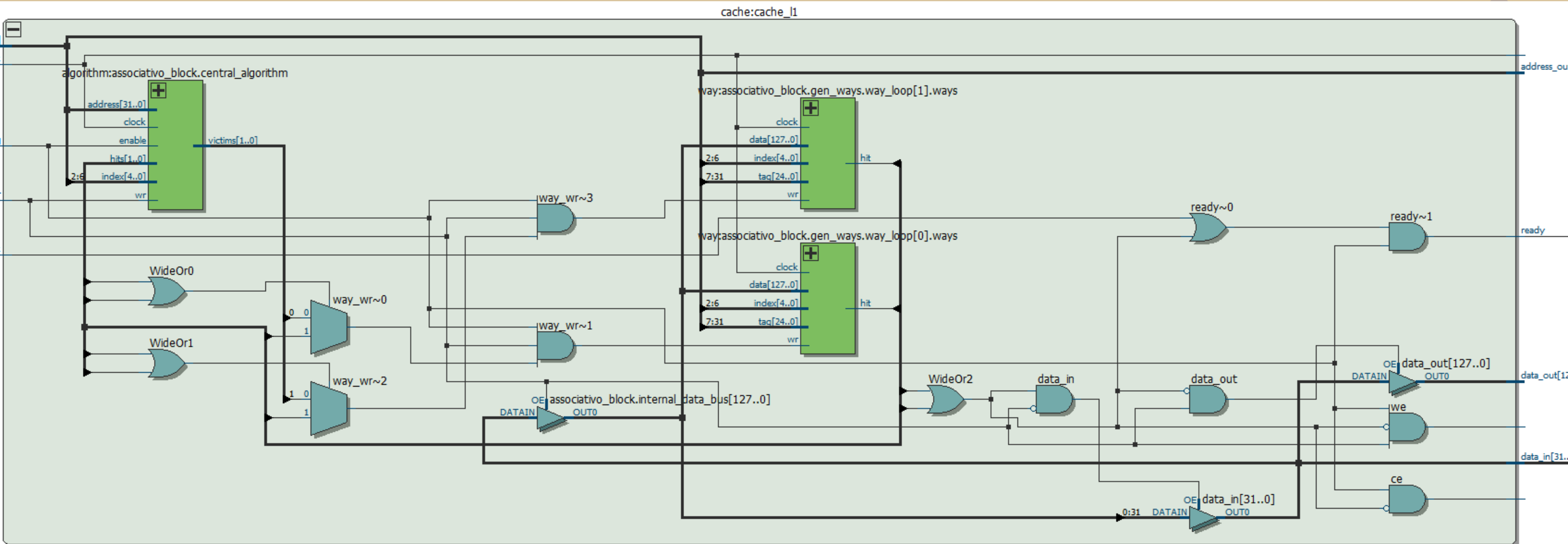
RTL DO QUARTUS II

- Além dos relatórios de área, frequência e caminho crítico o Quartus II também nos permite visualizar um diagrama rtl do código sintetizado, permitindo visualizar o hardware construído.



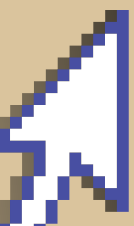
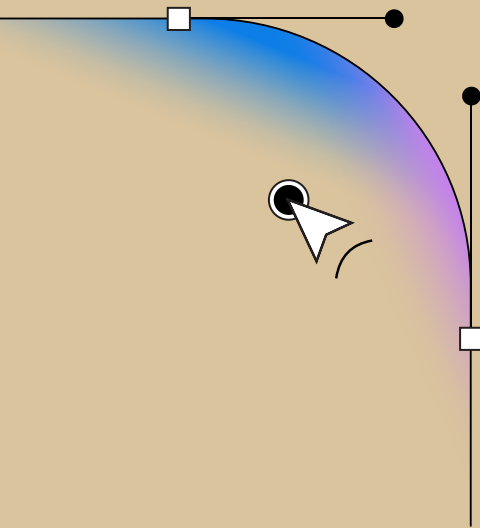
RTL DO QUARTUS II

- E nesse RTL também podemos ir expandindo os blocos para visualizar a forma como foram implementados pelo programa.



CONCLUSÃO

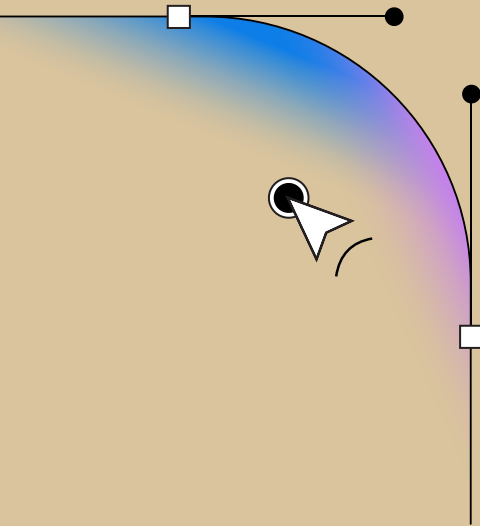
- A síntese lógica no FPGA Cyclone III provou a viabilidade prática de implementar o perceptron diretamente em nível de hardware (RTL).
- O preditor neural superou o LRU tradicional em cenários de saltos estruturados, alcançando um ganho máximo de +7,32% na taxa de acerto global no benchmark Matrix Transpose.
- O paralelismo cobrou um custo alto. Houve um aumento substancial de área (+147,3% de Elementos Lógicos na configuração Perceptron-Perceptron) e uma queda acentuada na frequência máxima Fmax de 90,92 MHz para 28,93 MHz.
- A configuração LRU na L1 + Perceptron na L2 mostrou-se a escolha ideal, entregando a maior parte do ganho preditivo (+6,50%) sem penalizar drasticamente a área e o clock do sistema.



REFERÊNCIAS

JIMÉNEZ, D. A.; LIN, C. Dynamic Branch Prediction with Perceptrons. Department of Computer Sciences, The University of Texas at Austin, 2001.

TERAN, E.; WANG, Z.; JIMÉNEZ, D. A. Perceptron Learning for Reuse Prediction. Texas A&M University / Intel Labs, IEEE, 2016.



OBRIGADO!

